

Group charter — Grimdark Forge (COSC3046, updated for Assessment Task 2)

Project	Grimdark Forge — miniature wargaming store and community
Student	Augustus Zibell-Barnes (S4246254)
Modules	Discussion Forum, Shopping Cart, Blog, Product Review & Rating

1. Roles and module ownership

The brief allocates one module per team member. This site is delivered by just me, a single student, so all four module leads are held by the same person. The modules are still kept as separate concerns — separate folders, separate page sets, and a separate accessibility self-audit statement each — so that each one can be assessed against the module criteria on its own terms.

Module	Folder	Lead
Discussion Forum	forum/	Augustus Zibell-Barnes
Shopping Cart	shop/	Augustus Zibell-Barnes
Blog	blog/	Augustus Zibell-Barnes
Product Review & Rating	reviews/	Augustus Zibell-Barnes

2. Ownership of the shared site shell

Decision. The site shell — the skip link, `<header>` with the wordmark, the primary `<nav>`, the header utility area (locale toggle, cart link, account link and the login/logout control), and the `<footer>` with its footer `<nav>` — is a single shared artefact. It is **not** owned by any one module.

Assessment Task 2 status: built server-side. The shell now lives in one place — `shared/header.php`, `shared/nav.php` and `shared/footer.php` — and every page pulls it in at request time with `require`. Nothing is copy-pasted any more; a change to any of those three files appears on every page of every module. The relative path prefix (`../` inside a module folder) is computed by `gf_base()` in `shared/bootstrap.php`, so no page carries its own copy of the shell and the site runs unchanged in any web root.

Responsibility. Whoever changes the shell is responsible for making the change on every page in the same commit, and for the shell continuing to meet these accessibility requirements on every page regardless of which module the page belongs to:

- **Landmarks.** `<header>`, `<nav>`, `<main id="main">` and `<footer>` are present on every page and are real landmark elements, not `<div>`s. Where a page carries more than one `<nav>`, each has a distinct accessible name via `aria-label` (`Primary`, `Breadcrumb`, `Footer`, and pagination names such as `Thread list pages`).

- **Heading structure.** The shell contributes no headings, so each page keeps exactly one `<h1>` in `<main>` and an unbroken heading sequence beneath it.
- **Focus visibility.** The focus ring is defined once in `assets/css/base.css` and is never removed. No rule in any stylesheet sets `outline: none` or `outline: 0`. The skip link is the first focusable element on every page and becomes visible on first Tab.
- **Language declaration.** Every page opens with `<html lang="...">` set to the active locale (`en-AU` or `de-DE`).

Verification. Before any shell change is committed, the same scripted checks are run against every page as rendered by PHP, in both locales: correct `<html lang>`, one `<h1>` per page, no skipped heading levels, a `<label for="...">` for every form control, an `alt` attribute on every `<img>`, no duplicate `id` values within a page, no HTML validation attributes, no inline styles, every `aria-describedby` target present, and every internal link resolving.

Currently signed-in user. The shell carries the login/logout feature required by Assessment Task 2. When nobody is logged in the header shows a **Log in** button that leads to `account.php`, where a username (no password) is entered — or, for a new visitor, to `register.php` to create an account first; once logged in the header shows the account's avatar and name and a **Log out** button. The username is held in the PHP session, so the same user is identified on every page of every module. Content owned by that account shows Edit and Delete controls; everyone else's does not, and the modules' PHP refuses (HTTP 403) any edit or delete request for content the logged-in user does not own. This convention is shared by all four modules and changing it is a shell change, not a module change.

3. Locale-switching mechanism

Planned in Assessment Task 1, **implemented in Assessment Task 2 exactly as agreed**, with the details confirmed below each original decision.

- **Control.** A single locale toggle lives in the shared header utility area, to the left of the cart link, in the same position on every page. *Confirmed:* it is rendered once, in `shared/header.php`, in the position the placeholder occupied.
- **Form.** A `<button>` showing a globe icon and the current locale's short name (for example "English"), carrying an `aria-label` written as an ordinary sentence rather than a code-like string, because the icon alone gives no accessible name. When it becomes functional it expands to a list of locales; the selected one is marked with `aria-current="true"`. *Confirmed:* the `<button>` carries `aria-expanded` and `aria-controls`; the list is a `<ul>` of submit buttons inside a `<form method="post">`, so it also works with JavaScript disabled (revealed on focus/hover by CSS). `assets/js/site.js` adds open/close on click, Escape and outside-click, and moves focus into the list.
- **Scope.** The toggle switches the whole site, not one module. Choosing a locale on a forum page keeps that locale when the reader moves to the shop. *Confirmed:* the choice is posted to `shared/locale.php`, stored in `$_SESSION['locale']`, and read by every page through `locale()`.
- **Persistence.** The choice is stored against the PHP session (in memory for the duration of the visit, as the brief requires) and reflected by the `<html lang="...">` attribute on every page, so assistive technology announces the correct language. *Confirmed:* `<html lang="<?= locale() ?>">` in `shared/header.php`.
- **Locales supported.** `en-AU` (English, Australia — the source language) and `de-DE` (German). UI strings are written in English in the templates and passed through `t()`; `shared/lang/de-DE.php` maps all 747 of them to German. A missing translation falls back to English rather than to a blank.

- **Locale-aware formatting.** Every date, price and count goes through `fmt_date()`, `money()` and `number()` in PHP (ICU via the `intl` extension when present, a manual table otherwise) and is wrapped so that `assets/js/site.js` re-formats it with `Intl.DateTimeFormat` / `Intl.NumberFormat` for the same locale. Nothing is hard-coded to an English/Australian pattern.
- **Consequences accepted now.** So that the switch is possible later, this assessment already avoids baking any meaningful text into images (all SVGs are text-free), writes `alt`, `aria-label` and `aria-describedby` values as ordinary translatable sentences, and keeps dates in `<time datetime="...">` and prices as marked-up text so their presentation can be re-formatted per locale rather than being hardcoded.

Status. Working. Choose *Deutsch (Deutschland)* from the header toggle on any page: the navigation, buttons, headings, form labels, hints and validation messages switch to German, prices render as `42,00 AU$`, dates as `13. August 2026 um 21:40`, and `<html lang="de-DE">` is set. Choose *English (Australia)* to switch back.

4. Working as a team in Assessment Task 2

Progress was committed to `main` throughout the assessment period rather than in one push. Each module was built against the shared shell rather than around it: the shell and the shared validation, image and formatting helpers were written first, then each module's five pages, then the German dictionary once every UI string existed. The module pages call the same helpers (`t()`, `money()`, `time_html()`, `validate()`, `image_fields_html()`), which is what keeps the four modules behaving identically.

5. Development and production plan

1. Agree the theme, module split and page inventory; write the wireframes.
2. Build the shell and the three stylesheets (`base.css` design tokens, `layout.css` regions, `components.css` reusable pieces) before any module page, so every module inherits the same structure and no module needs its own stylesheet.
3. Build each module's five pages against the wireframes, reusing existing components in preference to adding new ones.
4. Run the scripted accessibility and link checks in section 2 after each module.
5. Write each module's self-audit statement, naming a genuine remaining gap rather than a generic one.
6. Deploy to the RMIT core teaching servers and re-check every page there, since all marking is carried out from that environment.

6. Working agreements

- Commit to `main` regularly across the assessment period rather than in one final push, so progress is reviewable.
- Raw HTML, CSS, JavaScript and PHP only; no database yet (Assessment Task 3), and no third-party CSS or JS framework. No inline `style` attributes — all presentation lives in the three stylesheets. No HTML validation attributes — validation is JavaScript and PHP only.
- No page is named `index` or `home`.
- All paths are relative, so the site can be moved between servers unchanged.

- No remote requests: fonts fall through to the platform UI stack and all images are stored locally, so the site renders identically with the network disconnected.

7. Assessment Task 2 working constraints (formal record)

The Summary Report lists the charter changes as bullet points; this section records each as a binding agreement so that any later work (including Assessment Task 3) is measured against it.

7.1 Shared shell — implementation agreement

Item	Agreement
Location	<code>shared/header.php</code> (document head, <code><header></code>), locale toggle, cart count, login/logout), <code>shared/nav.php</code> (primary <code><nav></code>), <code>shared/footer.php</code> (<code><footer></code> and its <code><nav></code>).
Inclusion	Every page starts with <code>require_once __DIR__ . '/../shared/bootstrap.php'</code> (or <code>/shared/bootstrap.php</code> at the root) and then <code>require s</code> header and footer. No page may embed its own copy of any shell markup.
Paths	<code>gf_base()</code> in <code>shared/bootstrap.php</code> derives the <code>../</code> prefix from the file system, so the shell works from any web root and any module folder without editing.
Per-page data	A page sets <code>\$page_title</code> and <code>\$page_description</code> before requiring the header; the header owns <code><title></code> , <code><meta name="description"></code> and <code><html lang></code> .
Session state	<code>session_start()</code> happens once, in the bootstrap. The shell reads <code>current_user()</code> and <code>locale()</code> ; it never writes to the session directly. Login/logout are handled by <code>account.php</code> ; the locale by <code>shared/locale.php</code> .
Change rule	A shell change is any edit to the three shell files, to <code>gf_base()</code> , to <code>current_user()</code> / <code>owns()</code> / <code>login()</code> / <code>logout()</code> , or to the locale helpers. It is committed on its own with the rendered-page checks in section 2 re-run across all 23 pages in both locales.

7.2 Locale-switching — agreements confirmed

- The toggle is a `<form method="post" action=".../shared/locale.php">` so it functions without JavaScript; `assets/js/site.js` only improves it (open/close, Escape, outside click, focus management).
- `shared/locale.php` accepts only the tags listed in `shared/i18n.php`, stores the choice in `$_SESSION['locale']`, and redirects back to the page the visitor came from using `safe_return()` (same-site relative paths only, never an arbitrary URL).
- Every UI string passes through `t()` (or `te()` when it is also HTML-escaped). Adding a string to a template without a matching entry in `shared/lang/de-DE.php` is accepted temporarily — the fallback is English — but the dictionary is completed before the module is considered finished. The dictionary currently holds 747 entries.
- All dates, prices and counts use `fmt_date()` / `time_html()`, `money()` / `money_html()` and `number()`. Writing `date()`, `number_format()` or a literal `$` sign into a template is a defect.

7.3 New constraints introduced by Assessment Task 2

1. **Storage.** No database and no files under the web root. All data lives in the associative arrays seeded by `shared/data.php` and kept alive between requests by `shared/store.php` (APCu shared memory, or a locked

scratch file in the system temp directory when APCu is absent). Only per-visitor state (login, locale, guest cart, CSRF token, flash messages) lives in the PHP session.

2. **Input handling.** All request data is read through `filter_input()` (never `$_GET` / `$_POST` directly); every POST form carries `csrf_field()` and every POST handler checks `csrf_ok()` before touching the store; every value echoed into HTML goes through `e()`.
3. **Validation twice.** Rules are declared once per form in PHP (`$rules` arrays) and mirrored as `data-*` attributes for `assets/js/validate.js`. The server result is authoritative. No HTML validation attribute (`required`, `pattern`, `min`, `max`, `maxlength`, `type="email"` for validation purposes) may appear in any template.
4. **Ownership.** Edit and delete controls render only when `owns($author)` is true, and the handler pages return HTTP 403 (or 404 for a missing record) for anyone else. The check is in the PHP, not only in the template.
5. **Soft deletion in the forum.** Forum posts are never removed from the array; `deleted => true` hides them. Other modules delete outright, as their briefs allow.
6. **Images.** Members may attach a library image or upload one (JPEG/PNG/GIF/WebP, at most 1 MB, verified with `getimagesize()` rather than the client-supplied MIME type). Uploads are stored as data URIs inside the store so no file is written under the web root. A text alternative is mandatory whenever an image is attached.
7. **Money.** All prices are integers in cents. Totals, discounts, delivery and GST (total $\div$ 11) are computed server-side in `cart_totals()`; `assets/js/shop.js` only mirrors them for live display.
8. **Compatibility.** `php -l` on every file before commit; no PHP 8-only syntax so the site runs on the CoreTeaching servers' PHP 7.4.

7.4 Definition of done for a module page

A page is done when: it renders in both locales with no PHP notices; the section 2 checks pass; its create/update/delete paths have been exercised over HTTP with valid, invalid and foreign-owner requests; and the German dictionary contains every string it introduced.